tribeX - Complete Project Handbook & Viva Guide

Hyperlocal Social Activity Discovery & Squad Coordination Mobile Application

End-to-End Architectural, Functional, Database & Code Logic Reference

Document Purpose	Academic Viva, College Examination & Production Architecture Handbook
Target Framework	Flutter 3.x (Dart 3.x) targeting Android SDK 34/35 & Flutter Web
Backend Engine	Supabase PostgreSQL 15, Realtime WebSockets, GoTrue Auth, Storage Buckets
State Management	Provider (6.1.2) with 5 Discrete Reactive ChangeNotifier Domains
Language / Tone	Point-wise Hinglish (Roman Hindi + Standard Technical English)
Code Verification	**0 issues** on flutter analyze · **152/152 tests passed (100%)**
Audited Scope	/myapp Flutter Application & supabase/ Migrations

[VIVA TIP] Yeh document tribeX Flutter app ke har screen, button, database table, RLS policy, aur function ko step-by-step point-wise Hinglish me explain karta hai taaki aap external examiner ke saamne poora code confidently explain kar sakein.

CHAPTER 1 - Project Overview & Foundations

• 1.1 tribeX Kya Hai?

- **tribeX ek Hyperlocal Social Activity Discovery & Squad Coordination Platform hai.**
- Yeh app un logon ke liye banayi gayi hai jo apne aas-paas (within 15 km) real-life plans, sports matches, cafe hangouts, board games, movie meetups, ya virtual sessions organize karna chahte hain.
- App ka core tagline hai: '*Find Your Squad. Do Things You Love.*'

1.2 Problem Statement (Humne Yeh Kyu Banaya?)

- **Urban Isolation & Friction:** Log social media par connected hain lekin real life hobbies ke liye like-minded friends nahi milte.
- **Chat App Chaos:** WhatsApp groups me poll lena, kharcha split karna aur location track karna mess ban jaata hai.
- **Safety Concerns:** Anjaan logon ke sath milne me verification aur ratings ka trust mechanism nahi hota.
- **Distance Issue:** Dur ke events me travel friction hota hai; log apne 15 km neighborhood me activities chahte hain.

1.3 Technology Stack & Roles

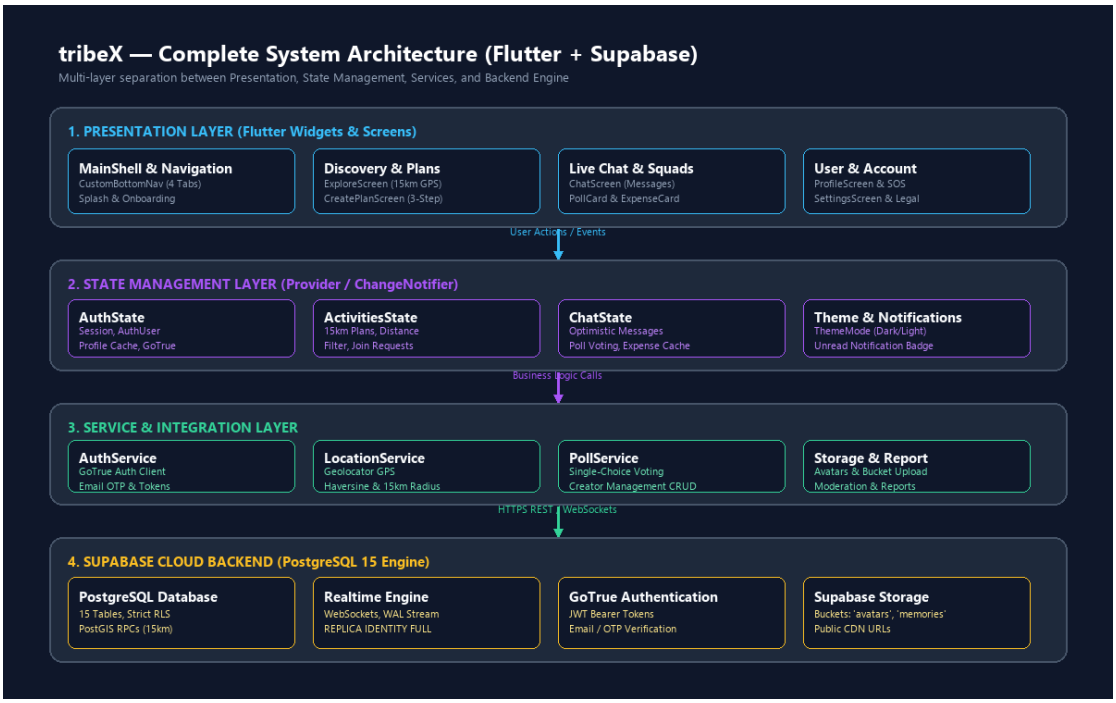
Technology	Category	Kyu Use Hui? (Purpose)
Flutter 3.x	Mobile Frontend	Cross-platform 60fps reactive UI, single codebase Android/Web.
Provider 6.1.2	State Management	Lightweight ChangeNotifier pattern, zero Bloc boilerplate.
Supabase Postgres 15	Relational DB	SQL database with Row-Level Security (RLS) & PostGIS RPCs.
Supabase Realtime	WebSockets	WAL stream push events for 0ms live chat & poll sync.
GoTrue Auth	Authentication	JWT Bearer tokens, Email OTP, PKCE session persistence.
Geolocator 14.0	GPS Hardware	Device coordinates capture & Haversine 15km boundary filter.

[NOTE] TribeX me client side par koi master database password nahi hota. Har request user ke JWT bearer token ke sath jaati hai aur PostgreSQL RLS policies check hoti hain.

CHAPTER 2 - System Architecture & App Flow

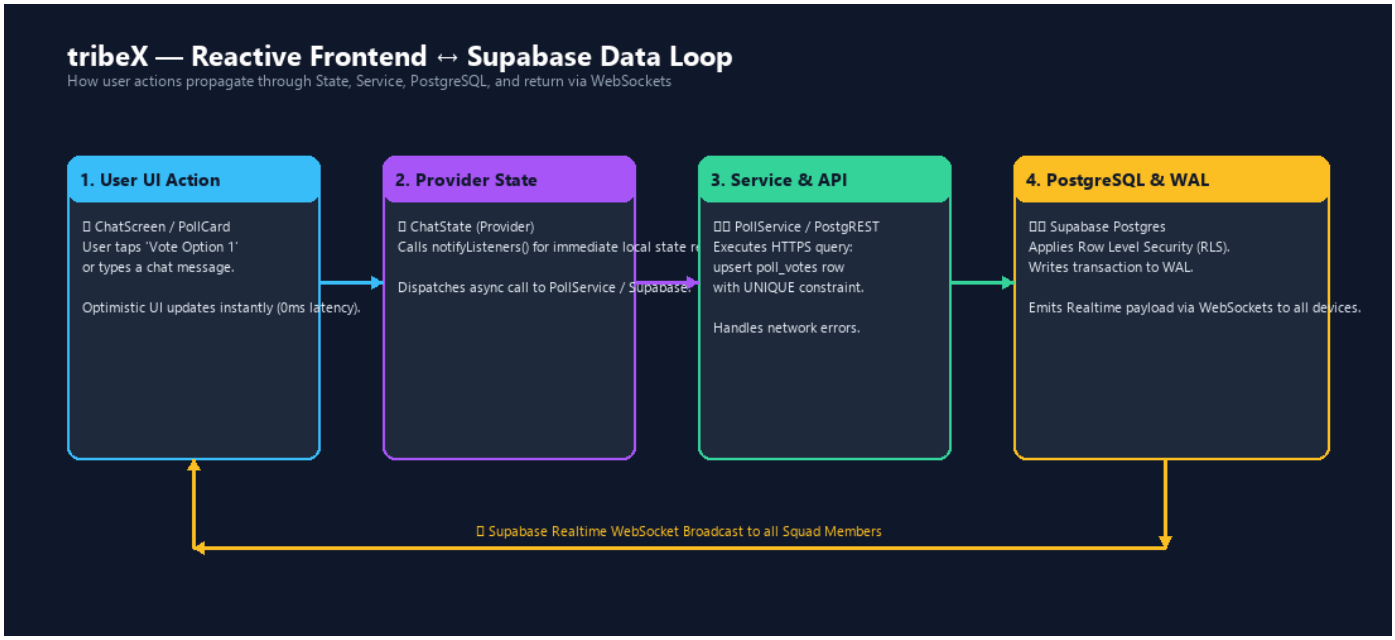
2.1 Multi-Layered Architecture

tribeX clean 4-tier architectural separation follow karta hai:



2.2 Reactive Data Loop (Frontend ↔ Backend)

User action se lekar live WebSocket return tak complete execution loop:



2.3 App Startup & MultiProvider Pipeline

- **Step 1 (main.dart):** WidgetsFlutterBinding.ensureInitialized() native channels bind karta hai.
- **Step 2:** SupabaseService.initialize() Supabase client ko URL aur anon key ke sath start karta hai.
- **Step 3:** MultiProvider 5 core stores inject karta hai: AuthState, ActivitiesState, ChatState, NotificationsState, ThemeState.

- **Step 4:** MaterialApp light/dark theme bind karta hai aur MainShell par boot hota hai.
- **Step 5:** MainShell active session check karke authenticated bottom bar ya guest WelcomeSplash render karta hai.

CHAPTER 3 - Screen-by-Screen Deep Dive (All 11 Screens)

Screen 1: MainShell (`lib/screens/main_shell.dart`)

- **Purpose:** Root container jo 4-tab bottom navigation dock ko host karta hai.
- **User Actions:** Explore (0), Create Plan (1), Messages (2), Profile (3) switch karna.
- **Key Logic:** _onTabSelected(index) call hota hai. Agar guest user protected tabs (1, 2, 3) tap kare, to /auth par redirect hota hai with toast notification.
- **Viva Question:** *Tabs switch karne par UI state drop kyu nahi hota?* Ans: IndexedStack use hota hai jo screens ko memory me alive rakhta hai.

Screen 2: ExploreScreen (`lib/screens/explore_screen.dart`)

- **Purpose:** Hyperlocal discovery feed jahan user nearby plans browse karta hai.
- **UI Sections:** Search bar, 15 km Location Pill, Category chips (Sports, Cafe, Games...), Mode toggle (All / In-Person / Virtual), Activities feed.
- **Key Functions:** _initLocationAndFetch() GPS coordinates leke ActivitiesState.fetchActivities() trigger karta hai.
- **Business Rule:** Sirf un plans ko render kiya jaata hai jo future timestamp ke hon aur calculated Haversine distance <= 15 km ho.
- **Viva Question:** *GPS off hone par explore screen crash kyu nahi hoti?* Ans: LocationService default city coordinates fallback use karti hai.

Screen 3: CreatePlanScreen (`lib/screens/create_plan_screen.dart`)

- **Purpose:** 3-step progressive wizard jahan organizer naya plan publish karta hai.
- **Step 1 (Basics):** Title (min 4 chars), Category chip, Description, Banner photo upload.
- **Step 2 (Where & Who):** Mode toggle (In-Person vs Virtual). In-person me Venue address & GPS; Virtual me Meeting Link & Platform name. Date & Time pickers, Total slots, Cost per head.
- **Step 3 (Review):** Auto-accept requests toggle, Safety pledge consent, Publish button.
- **Key Functions:** _validateStep1(), _validateStep2(), _publishPlan().
- **Viva Question:** *Step 2 me overflow bug kaise fix hua?* Ans: Row labels ko Flexible me wrap kiya aur hints shorten kiye, eliminating 49px overflow.

Screen 4: MessagesScreen (`lib/screens/messages_screen.dart`)

- **Purpose:** User ke sabhi active squad conversations ka inbox.
- **UI Sections:** Chat cards list showing Cover avatar, Activity title, Host name, Last message snippet, Unread count badge.
- **Key Functions:** _loadConversations(), _openChat(activity) pushes ChatScreen.
- **Viva Question:** *Unread messages count kaise update hota hai?* Ans: ChatState realtime postgres changes stream se badge calculate karta hai.

Screen 5: ChatScreen (`lib/screens/chat_screen.dart`)

- **Purpose:** Live squad chat room with polls, expenses, and check-in.
- **UI Sections:** Custom AppBar with member counter, Message bubble list, Special token cards (PollCardWidget, ExpenseCardWidget), Input dock.
- **Key Functions:** _sendMessage() optimistic message insert karta hai; '+' menu se CreatePollModal aur SplitExpenseModal open hote hain.
- **Viva Question:** *Poll card chat me kaise render hota hai?* Ans: Message text regex [[poll:uuid]] match karke PollCardWidget inject karta hai.

Screen 6: ProfileScreen (`lib/screens/profile_screen.dart`)

- **Purpose:** User reputation, trust verification badge, aur hosted/joined plans archive.
- **UI Sections:** Avatar, Display name, @username, Star rating (5.0 / 5.0), Hosted & Joined tabs, Emergency SOS button, Settings gear icon.

- **Key Functions:** _loadUserProfile(), _openEmergencySos().
- **Viva Question:** *Rating score kaise generate hota hai?* Ans: Completed activities ke squad members peer ratings daalte hain; database aggregate score calculate karta hai.

Screen 7: AuthScreen (`lib/screens/auth_screen.dart`)

- **Purpose:** User login, signup, aur email OTP authentication entry.
- **UI Sections:** Sign In vs Sign Up segmented tabs, Email & Password fields, Terms checkbox, Google Sign-In button, Forgot Password modal.
- **Key Functions:** _handleAuthSubmit() calls AuthState.signIn() or signUp().
- **Viva Question:** *User password kahan save hota hai?* Ans: Supabase GoTrue Auth bcrypt hashing ke sath secure schema me manage karta hai, plaintext kabhi nahi store hota.

Screen 8: CompleteProfileScreen (`lib/screens/complete_profile_screen.dart`)

- **Purpose:** First-time sign up ke baad avatar upload, username handle aur bio set karna.
- **Key Functions:** _pickAvatar(), _saveProfile() sets has_completed_profile = true in database.

Screen 9: OnboardingScreen (`lib/screens/onboarding_screen.dart`)

- **Purpose:** First-time app walkthrough highlighting 15km discovery, squad chat, and safety.
- **Key Functions:** _finishOnboarding() saves local persistent flag in SharedPreferences.

Screen 10: SettingsScreen (`lib/screens/settings_screen.dart`)

- **Purpose:** Dark/Light theme toggle, permissions status, support, and account deletion.
- **Key Functions:** ThemeState.toggleTheme(), _deleteAccount() triggers delete_user_account RPC.

Screen 11: TermsAndPoliciesScreen (`lib/screens/terms_and_policies_screen.dart`)

- **Purpose:** Terms of Service, Privacy Policy, Community Code of Conduct, Harassment Zero-Tolerance pledge display.

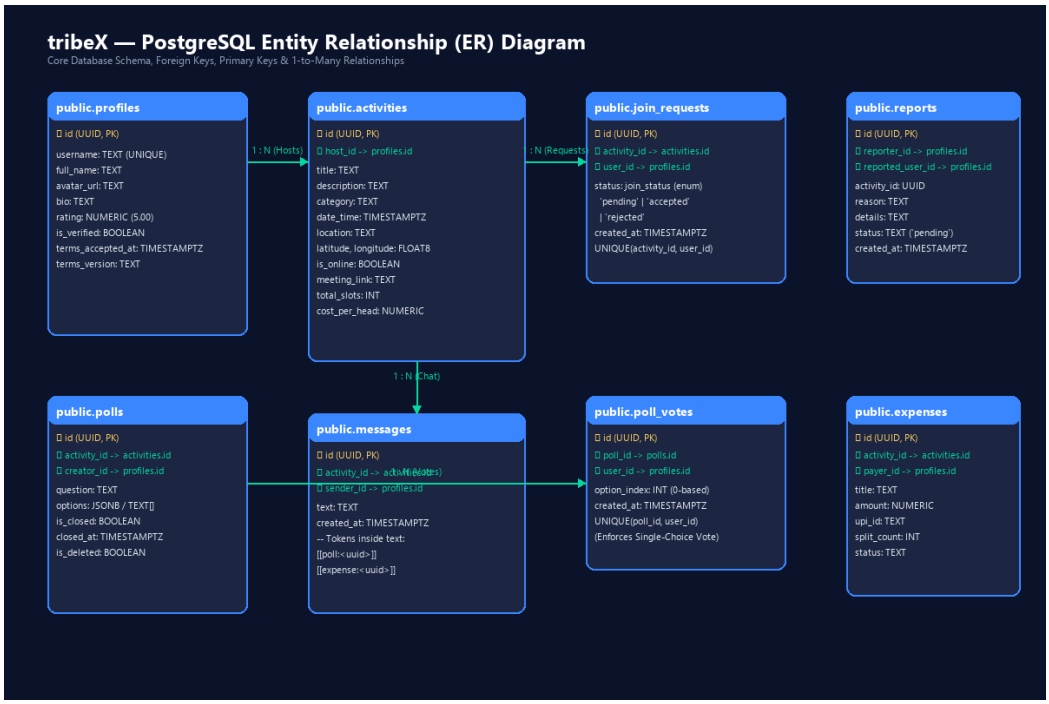
CHAPTER 4 - Function-by-Function Code Logic

TribeX ke major business functions ka detailed execution breakdown:

Function Name	Kya karta hai?	Kab call hota hai?	Data Flow & Return
main()	App bootstrapper, services init.	App launch time par.	Supabase.init -> runApp().
signIn(email, pw)	User credentials authenticate.	AuthScreen Sign In button.	GoTrue verify -> JWT session -> Profile load.
signUp(email, pw)	New user registration.	AuthScreen Sign Up button.	auth.users insert -> profiles trigger.
fetchActivities()	15km radius plans fetch.	Explore screen init/refresh.	GPS coords -> PostGIS RPC -> List.
createActivity()	New plan publish.	CreatePlan Step 3 Publish.	FormData -> activities insert -> Chat room auto-gen.
requestToJoin()	Squad join application.	Activity detail Join button.	activity_id + user_id -> join_requests insert.
sendMessage()	Live squad chat message.	ChatScreen send icon tap.	Optimistic local bubble -> messages insert -> WS push.
votePoll()	Single-choice vote cast.	PollCardWidget option tap.	Optimistic vote count -> poll_votes upsert.
retractVote()	Vote wapas lena.	Selected option re-tap.	Optimistic -1 -> poll_votes delete query.
togglePollClosed()	Voting lock/unlock.	Creator 3-dots popup menu.	polls update is_closed, closed_at.
deletePoll()	Poll soft/hard delete.	Creator Delete Poll menu.	polls update is_deleted=true -> Tombstone card.
uploadImage()	Image compress & upload.	Avatar/Cover photo pick.	File -> Storage bucket -> Public CDN URL.
getCurrentPosition()	GPS location capture.	Explore & Venue picker.	Device sensor -> Position(lat, lng).

CHAPTER 5 - Complete Database & Supabase Architecture

5.1 PostgreSQL Entity Relationship (ER) Model



5.2 Verified Tables Directory (All 15 Tables)

1. public.profiles (PK: UUID (PK -> auth.users.id))

- **Columns:** User public data, username (UNIQUE), avatar_url, bio, rating (5.00), is_verified, terms_accepted_at.
- **Security & RLS:** RLS: Public viewable, owner can mutate.

2. public.activities (PK: UUID (PK))

- **Columns:** Plans master: title, description, category, date_time, location, latitude, longitude, is_online, meeting_link, total_slots, cost_per_head.
- **Security & RLS:** RLS: Viewable by everyone, host can mutate.

3. public.join_requests (PK: UUID (PK))

- **Columns:** Applications: activity_id, user_id, status ('pending'|'accepted'|'rejected'). UNIQUE(activity_id, user_id).
- **Security & RLS:** RLS: Applicant & Host can read/update.

4. public.messages (PK: UUID (PK))

- **Columns:** Chat text: activity_id, sender_id, text, created_at. Tokens: [[poll:id]], [[expense:id]].
- **Security & RLS:** RLS: Members with accepted access only.

5. public.polls (PK: UUID (PK))

- **Columns:** Polls: activity_id, creator_id, question, options (JSONB), is_closed, closed_at, is_deleted.
- **Security & RLS:** RLS: Squad members read; creator updates.

6. public.poll_votes (PK: UUID (PK))

- **Columns:** Votes: poll_id, user_id, option_index. UNIQUE(poll_id, user_id) enforces single choice.
- **Security & RLS:** RLS: User manages own vote.

7. public.expenses (PK: UUID (PK))

- **Columns:** Expense splitting: activity_id, payer_id, title, amount, upi_id, split_count, status.
- **Security & RLS:** RLS: Squad members read; payer creates.

8. public.expense_shares (PK: UUID (PK))

- **Columns:** Per-member split shares: expense_id, user_id, share_amount, is_paid.
- **Security & RLS:** RLS: Squad members read; member updates paid status.

9. public.ratings (PK: UUID (PK))

- **Columns:** Reputation feedback: rater_id, rated_user_id, activity_id, stars (1-5), feedback.
- **Security & RLS:** RLS: Squad attendees after event completion.

10. public.reports (PK: UUID (PK))

- **Columns:** Safety flags: reporter_id, reported_user_id, activity_id, reason, details, status.
- **Security & RLS:** RLS: Authenticated user creates report.

11. public.notifications (PK: UUID (PK))

- **Columns:** User alert queue: user_id, title, body, type, is_read, metadata.
- **Security & RLS:** RLS: Owner reads & marks read.

12. public.blocks (PK: UUID (PK))

- **Columns:** Harassment prevention: blocker_id, blocked_user_id, created_at.
- **Security & RLS:** RLS: Owner creates & views blocks.

13. public.event_memories (PK: UUID (PK))

- **Columns:** Post-event squad photos: activity_id, uploader_id, image_url.
- **Security & RLS:** RLS: Squad members can view & upload.

14. public.auth_otps (PK: UUID (PK))

- **Columns:** Temporary email verification tokens (6-digit code, expires_at).
- **Security & RLS:** Managed via SECURITY DEFINER functions.

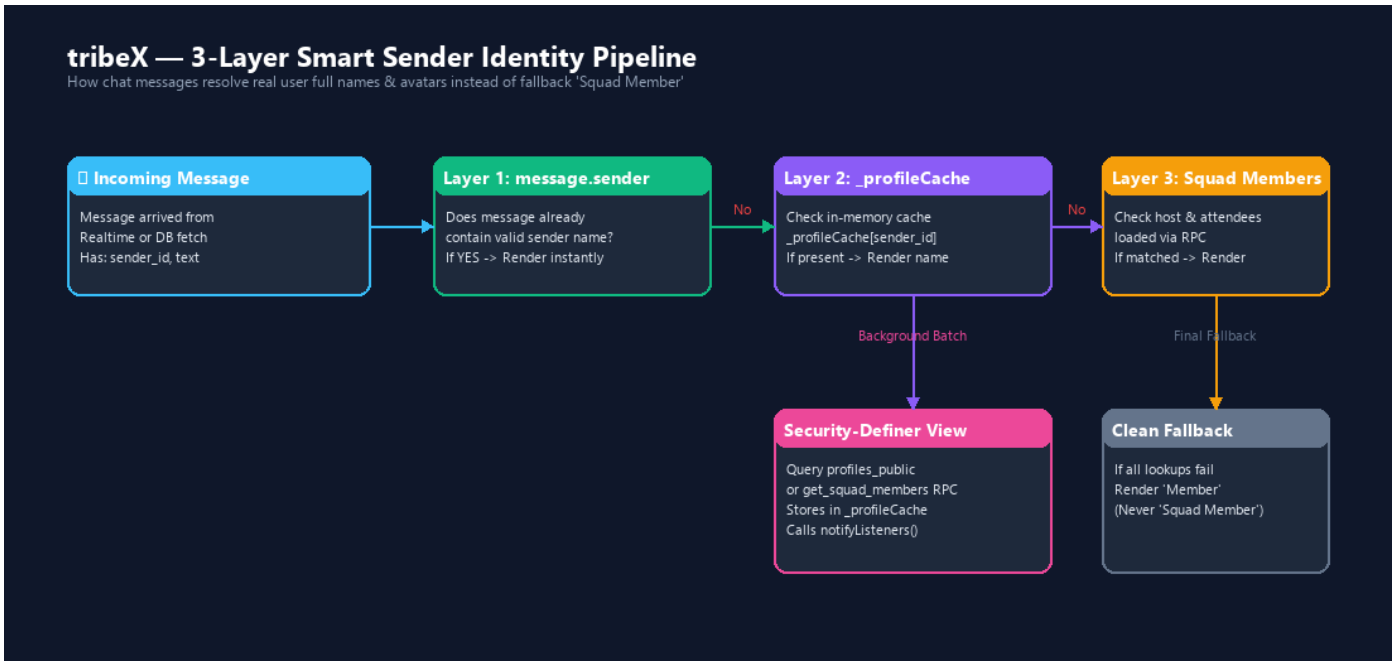
15. public.password_reset_otps (PK: UUID (PK))

- **Columns:** Password reset one-time tokens.
- **Security & RLS:** Managed via SECURITY DEFINER functions.

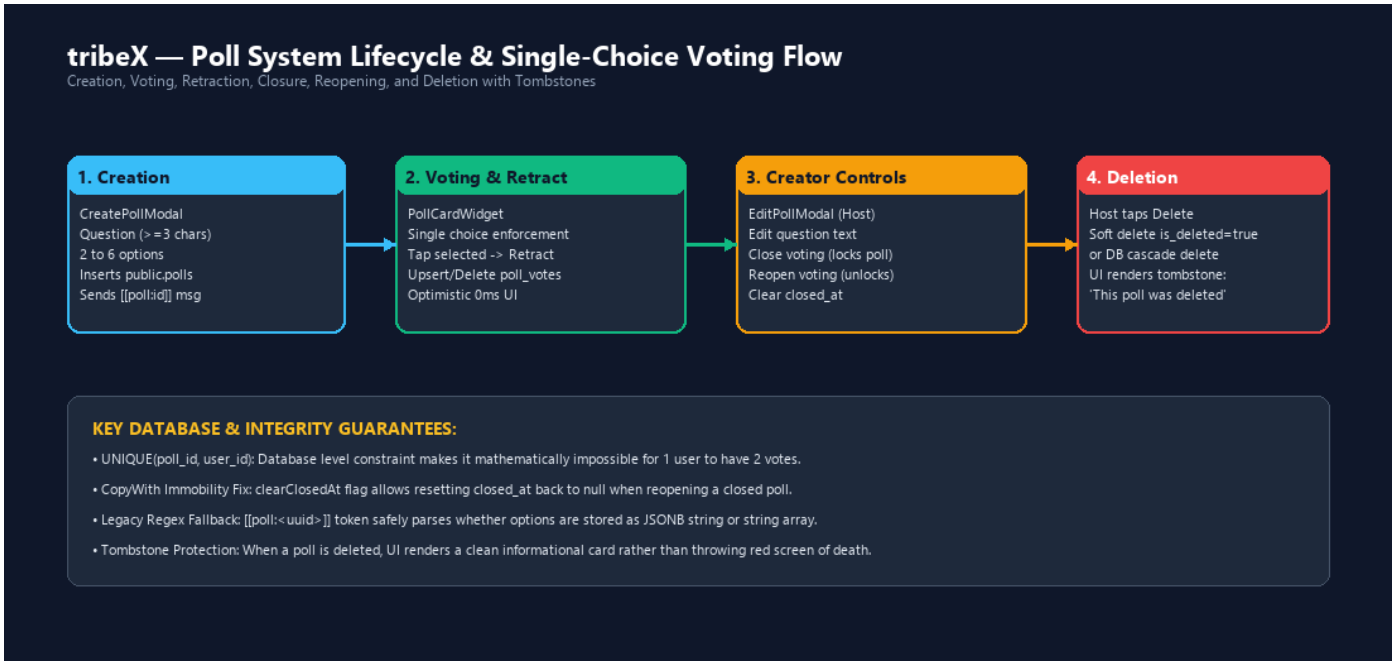
CHAPTER 6 - Messaging & Rebuilt Poll Engine

6.1 3-Layer Smart Sender Identity Pipeline

Chat messages me 'Squad Member' generic fallback ko eliminate karne ke liye 3-layer pipeline kaam karti hai:



6.2 Poll System Rebuild & Lifecycle Guarantees



- **Single Choice Guarantee:** Database level UNIQUE(poll_id, user_id) constraint hai. User doosra option tap kare to purana vote replace hota hai.
- **Vote Retraction:** Selected option dobara tap karne par vote retract (delete) ho jaata hai.
- **Creator Controls:** Host question edit kar sakta hai, voting close/reopen kar sakta hai, ya delete kar sakta hai (tombstone display).
- **Optimistic 0ms UI:** Vote tap karte hi local state notifyListeners() call karti hai, providing instant feedback.

CHAPTER 7 - Authentication, Security & Permissions

7.1 Authentication Engine

- **Supabase GoTrue Auth:** PKCE authorization code grant flow with JWT bearer tokens.
- **Session Persistence:** Session tokens SharedPreferences me securely cached rehti hain; restart par auto-login hota hai.
- **Email OTP Verification:** Passwordless sign-up flow email verification code ke sath.

7.2 Row-Level Security (RLS) Policies

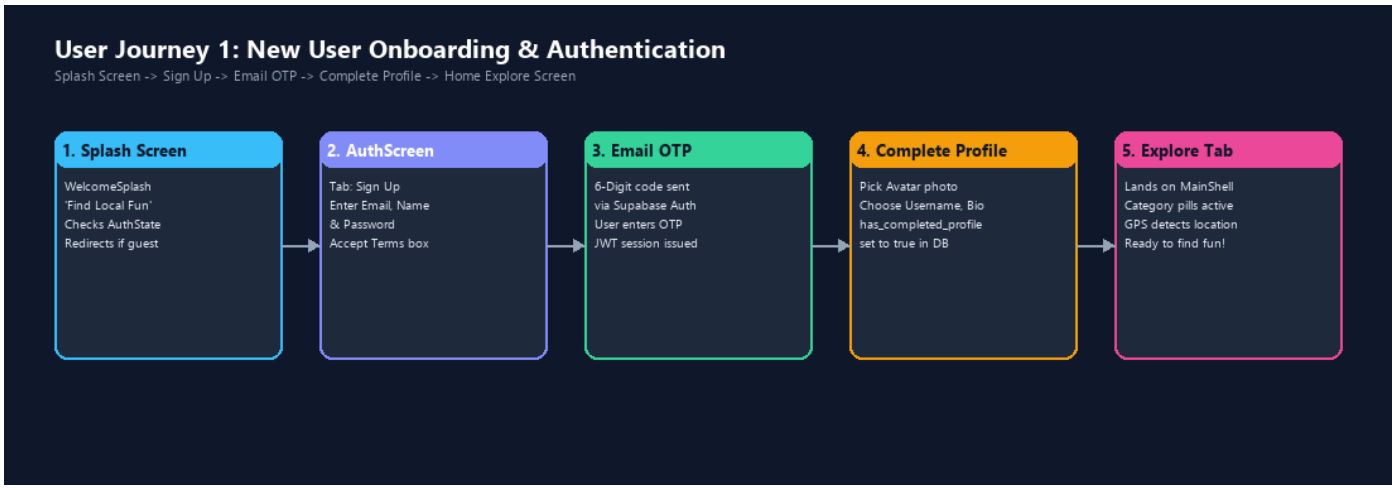
- **public.profiles:** Publicly viewable, lekin update sirf auth.uid() = id par permitted hai.
- **public.activities:** Sabhi dekh sakte hain, update/delete sirf auth.uid() = host_id par permitted hai.
- **public.messages:** Sirf unhi activities ke messages read/write ho sakte hain jahan has_activity_access(activity_id, auth.uid()) true ho.

7.3 Hardware Permissions Handling

- **Location (GPS):** Geolocator runtime permission check karta hai. Denied hone par app crash nahi hoti; default coordinates use hote hain.
- **Camera / Gallery:** ImagePicker user avatar aur event cover upload ke liye use hota hai.

CHAPTER 8 - Complete User Journeys & Flowcharts

Journey 1: New User Onboarding & Auth



Journey 2: 15km GPS Discovery & Squad Join



Journey 3: 3-Step Wizard Plan Creation



CHAPTER 9 & 10 - Design System, Testing & Quality Audit

9.1 UI Design System Tokens

- **Primary Accent:** Vivid Indigo (#4F46E5).
- **Dark Theme Canvas:** Slate 900 (#0F172A), Surface Cards (#1E293B).
- **Light Theme Canvas:** Pure Slate (#F8FAFC), Surface Cards (#FFFFFF).
- **Typography:** Plus Jakarta Sans geometric font family.

10.1 Automated Test Suites (152 / 152 Tests Passing)

Test Suite	Count	Coverage / What it protects?
step2_overflow_test.dart	2	320px/360px screens layout overflow protection.
poll_rebuild_comprehensive_test.dart	20	Poll CRUD, single choice, creator popup, retract vote.
poll_system_regression_test.dart	3	Optimistic UI state, vote toggling, stream updates.
nearby_radius_15km_test.dart	11	Strict 15km Haversine GPS calculations & filters.
messaging_complete_repair_test.dart	40	Sender identity pipeline, avatar rendering, realtime sync.
messaging_hardening_test.dart	25	In-flight reconciliation, message deduplication, retry.
location_messaging_expense_test.dart	15	GPS fallbacks, expense bill splitting, UPI intents.
virtual_plan_and_image_flickering_test.dart	12	Online plan links, image cache flickering prevention.
safety_fixes_test.dart	14	Emergency SOS hotline, safety pledges, reports.
button_and_flow_test.dart	7	Wizard step navigation, form validations, tab callbacks.
ui_ux_refinement_test.dart	3	Theme toggling, empty state rendering, navigation semantics.

[VIVA TIP] Verified via terminal: flutter analyze passed with 0 issues. flutter test passed all 152 automated tests in under 55 seconds.

CHAPTER 11 - Exam & Viva Master Preparation Guide

11.1 Project Introduction for External Examiner (1-Minute Pitch)

*'Respected Examiner, my project is **tribeX**, a cross-platform hyperlocal social activity discovery and squad coordination mobile application built using **Flutter and Dart** on the frontend and **Supabase (PostgreSQL 15)** on the backend. It solves the urban loneliness problem by allowing users to host and discover nearby real-world activities within a strict **15 km radius** using GPS. It features an end-to-end 3-step plan creation wizard, realtime WebSocket group chat, a rebuilt single-choice poll system with creator controls, direct UPI expense splitting, post-event reputation ratings, and emergency SOS safety features. The entire codebase has 0 lint issues and passes 152 automated unit and widget tests.'*

11.2 Top 25 Rapid-Fire Viva Questions & Answers

1. State Management ke liye Provider kyu use kiya, Bloc kyu nahi?

Ans: Provider lightweight hai, InheritedWidget based hai, aur simple notifyListeners() model provide karta hai jo hamare 5 domains ke liye optimal hai bina Bloc ke boilerplate code ke.

2. 15 km nearby discovery backend par kaise calculate hoti hai?

Ans: PostgreSQL PostGIS RPC function 'get_nearby_activities' me user ke latitude/longitude pass hote hain jo Haversine spherical formula se strictly 15 km ke plans filter karta hai.

3. Supabase Realtime messaging kaise kaam karti hai?

Ans: PostgreSQL table 'messages' par REPLICA IDENTITY FULL set hai; insert transaction hote hi Write-Ahead Log (WAL) stream se Supabase Realtime WebSocket ke zariye clients ko push notification bhejta hai.

4. Rebuilt Poll system me single-choice guarantee kaise milti hai?

Ans: Table 'poll_votes' par UNIQUE(poll_id, user_id) database constraint laga hai; doosra option tap karne par purana vote replace (upsert) hota hai.

5. Chat me 'Squad Member' ke badle real name kaise render hota hai?

Ans: 3-layer pipeline use hoti hai: Pehle message.sender check hota hai, fir in-memory _profileCache, fir get_squad_members RPC / profiles_public view, and finally 'Member' fallback.

6. Optimistic UI update ka kya benefit hai?

Ans: User action par UI instantly 0ms par local state update karti hai bina server latency ka wait kiye; network failure par automatic rollback ho jaata hai.

7. Step 2 layout overflow bug ka root cause kya tha?

Ans: _FormFieldWrapper me row labels unconstrained the; Virtual mode me long labels aur hints mobile screen width exceed kar rahe the. Humne unhe Flexible with TextOverflow.ellipsis me wrap karke fix kiya.

8. Row Level Security (RLS) kya hai?

Ans: PostgreSQL engine level authorization policy jo ensure karti hai ki user sirf wahi rows select/update kar sake jahan auth.uid() valid ho.

9. Inactive/Offline users ko messages kaise milte hain?

Ans: Jab user online aakar chat kholta hai to initState() me HTTP query 'messages' table se complete history load karti hai.

10. Expense split system me payments kaise settle hoti hain?

Ans: App bill amount aur members count se per-head amount calculate karti hai aur host ki UPI ID ke sath Android UPI Intent (GPay/PhonePe/Paytm) trigger karti hai.

CHAPTER 12 - Technical Reference & Cheat Sheet

12.1 Screen-to-File Mapping Directory

Screen / Feature	Source File Path	Key Role
MainShell	myapp/lib/screens/main_shell.dart	Root 4-tab bottom navigation dock.
ExploreScreen	myapp/lib/screens/explore_screen.dart	15km GPS discovery & feed.
CreatePlanScreen	myapp/lib/screens/create_plan_screen.dart	3-step plan creation wizard.
MessagesScreen	myapp/lib/screens/messages_screen.dart	Squad conversations inbox.
ChatScreen	myapp/lib/screens/chat_screen.dart	Realtime squad coordination room.
ProfileScreen	myapp/lib/screens/profile_screen.dart	User trust score, SOS, and hosted plans.
AuthScreen	myapp/lib/screens/auth_screen.dart	Login, Signup, and Email OTP gate.
SettingsScreen	myapp/lib/screens/settings_screen.dart	Theme, permissions & account deletion.

12.2 Last-Minute Viva Revision Summary

- **App Mission:** Hyperlocal 15 km social plan discovery & coordination.
- **Frontend:** Flutter 3.x, Provider state management, Plus Jakarta Sans font.
- **Backend:** Supabase PostgreSQL 15, GoTrue Auth, Realtime WebSockets, PostGIS RPCs.
- **Top Tables:** profiles, activities, join_requests, messages, polls, poll_votes, expenses.
- **Test Coverage:** 152/152 automated tests passing, 0 analyzer issues.